

BLOOD DONOR FINDER

BloodBridge — MERN Stack Web Application

PROJECT REPORT

1. Abstract

BloodBridge is a web-based blood donor discovery and blood-request management system developed using the MERN stack. The application provides a structured way for users to search available donors by recipient blood group and location, register as donors, and publish blood requests. The system is designed to reduce the time required to identify potential community donors during blood requirements.

2. Objectives

- Provide a centralized donor directory.
- Search donors using blood group compatibility and city.
- Allow donors to register and maintain availability information.
- Allow patients or attendants to publish blood requests.
- Provide a responsive and professional user interface.
- Demonstrate full-stack development using MongoDB, Express.js, React and Node.js.

3. Technologies Used

Layer	Technology
Frontend	React.js, Vite, JavaScript, CSS, Lucide React
Backend	Node.js, Express.js
Database	MongoDB, Mongoose
API	REST API
Development	VS Code, npm

4. System Modules

Donor Registration: Stores donor name, age, blood group, city, phone and optional email/last donation date.

Donor Search: Filters available donors using recipient blood group compatibility and city.

Blood Request: Records patient, required blood group, hospital, city, units, contact and urgency.

Request Listing: Displays open community requests for donor coordination.

Backend API: Express endpoints manage donor and request data.

5. Blood Group Matching

The application applies standard red-blood-cell compatibility rules as a software filtering aid. For example, an O+ recipient can be matched with O+ or O− donor groups in the application's search logic. The software does not replace hospital blood-bank testing, cross-matching, or medical advice.

6. System Architecture

React frontend → Express REST API → Node.js application layer → Mongoose → MongoDB. In demonstration mode, the backend uses sample in-memory data when MongoDB is not configured, allowing the application to

be demonstrated without a database server.

7. Functional Requirements

The system shall allow donor registration, donor search, compatibility filtering, donor contact actions, blood-request creation, and open-request viewing. The server validates required fields and provides JSON REST responses.

8. Non-Functional Requirements

The interface is responsive, simple to navigate, and designed for quick information retrieval. The API is separated from the frontend, making the project maintainable and extensible.

9. Testing Plan

Test Case	Expected Result
Open application	Home page loads successfully
Search O+ in Chennai	Compatible available donors in Chennai are displayed
Search with no matching city	No matching donors message is displayed
Register valid donor	Donor profile is created
Invalid donor age	Validation message is displayed
Create blood request	Request is published and listed
API health check	Server returns online/database status

10. Advantages

Fast donor discovery, centralized records, compatibility-based filtering, responsive design, clear REST architecture, and MongoDB readiness.

11. Limitations

The academic version does not independently verify donor identity, medical eligibility, hospital inventory, or real-time blood-bank stock. Contact information is supplied by users and should be handled responsibly.

12. Future Enhancements

Authentication and role-based access, verified donor profiles, hospital/admin dashboards, geolocation and maps, notification services, donor availability scheduling, audit logs, and secure production deployment can be added in future versions.

13. Conclusion

BloodBridge demonstrates a complete MERN-based solution for connecting blood donors and blood-request users. The modular architecture provides a practical foundation for extending the system into a larger verified healthcare-support platform.